



Smart Restaurant POS System

This project is a sophisticated Point of Sale (POS) system developed in **C#**, focusing on the application of **Object-Oriented Programming (OOP)** principles to ensure code robustness, maintainability, and scalability. The system automates the restaurant workflow from item selection and dynamic pricing to final invoice generation and payment processing.



Technical Class Descriptions

The system architecture is built according to the project's structural diagram. Below is a detailed description of each class and its functional role:

1. Product Hierarchy (Inheritance & Abstraction)

- **Product (Abstract Base Class):** The fundamental blueprint for all menu items. It stores shared attributes like **Name**, **Price**, and **Category**, and defines the **CalculatePrice()** virtual method to ensure all derived classes implement their specific pricing logic.
- **Food (Derived Class):** Represents solid meals. It extends the **Product** class by adding a **Calories** property and overrides the **CalculatePrice()** method to include calorie-based surcharges and manage item-specific Add-ons.
- **Drink (Derived Class):** Represents beverages. It includes a property for the drink type (Hot/Cold) and calculates additional costs based on temperature requirements (e.g., refrigeration for cold drinks).
- **Dessert (Derived Class):** Represents sweet items. It tracks **SugarGrams** and applies a pricing model influenced by sugar content, adhering to the project's specialized pricing rules.

2. Supporting Components

- **AddOn:** A dedicated class representing optional extras that can be added to products (e.g., extra cheese, sauce). It contains its own **Name** and **Price**, allowing for modular customization of individual order items.

3. Order Management (Business Logic)

- **OrderItem:** Acts as a container for a specific line in an order. It links a **Product** object with a **Quantity** and a collection of **AddOn** objects, calculating the **LineTotal** for that specific entry including all selected extras.
- **Order:** The central manager of the transaction. It maintains the **List<OrderItem>**, provides methods to add/remove/update items, and calculates the final subtotal including taxes and discounts.
- **RestaurantSettings (Static Class):** A utility class containing global constants such as **Tax Rate (10%)** and **Service Charge (5%)** to ensure consistent calculations across the system.

4. Payment System (Interfaces)

- **IPaymentMethod (Interface):** Defines the contract for all payment types, allowing the system to process payments polymorphically.

- **CashPayment:** Processes physical currency, validates the amount received, and calculates the precise "Change" for the customer.
- **CardPayment:** Simulates a secure card transaction, including logic to **mask** the card number (e.g., **** * 1234) for privacy and data protection.

 Pricing Formulas

Category	Calculation Formula	Technical Justification
Food	BasePrice + (Calories * 0.001) + Sum(AddOns)	Links cost to nutritional density and extras.
Drink	BasePrice + (IsCold ? 0.050 : 0)	Covers refrigeration overheads for cold items.
Dessert	BasePrice + (SugarGrams * 0.002)	Pricing model based on ingredient specifications.

 Key Technical Features

- **Robustness (Validation):** Implements input validation (e.g., `TryParse`) and logical constraints (e.g., restricting discounts to 0-100%) to prevent runtime crashes.
- **Data Persistence (File I/O):** Supports exporting final invoices to text files (`.txt`) for digital record-keeping and archiving.
- **Add-on System:** A flexible system that allows optional components for each order with real-time price updates.

 Sample Invoice Output

```
=====
                        Smart Restaurant POS
=====
No.   Item                               Qty      Unit      Total
-----
1     Classic Burger                     2        3.400     6.800
      Add-ons: Extra Cheese (0.250 KD)
-----
Subtotal:                               6.800 KD
Final Total:                             7.854 KD
-----
Payment Method: Card (No: **** * 1234)
=====
```

Developed as part of an intelligent automation systems initiative, focusing on high-quality software design and performance.